Errori dei tool MCP: progettare risposte che l'agente sa gestire

> Appunti per la certificazione CCA-F (Dominio 2). Spiegano come un tool deve **classificare e restituire gli errori**, **chi deve fare i retry** e come l'agente deve **reagire** a ciascun tipo di errore.
>
> ⚠️ I nomi dei campi come `errorCategory` o `isRetryable` non sono uno standard del protocollo: sono convenzioni di progettazione che compaiono spesso nelle domande. Lo standard MCP definisce solo il flag `isError` nel risultato del tool.

Indice

1. Perché gli errori contano
2. Errori di protocollo vs errori del tool
3. Le categorie di errore
4. Anatomia di una buona risposta di errore
5. Chi fa il retry: tool o agente?
6. Il caso più insidioso: "nessun risultato"
7. Successo apparente: `isError: false` ma dati inaffidabili
8. Il ciclo lato API: `tool_result` e `is_error`
9. Trappole e domande da esame

---

1. Perché gli errori contano

Per un agente, un errore è **informazione**. Il modello decide il passo successivo leggendo il risultato del tool: se l'errore è vago, decide male.

Una risposta di errore ben progettata permette all'agente di:

- **riprovare** quando ha senso (e solo allora);
- **correggere** la richiesta (parametro sbagliato, identificativo diverso);
- **chiedere** all'utente l'informazione mancante;
- **scalare** a un umano quando non c'è altra strada;
- **comunicare** all'utente la situazione reale, senza inventare.

> 💡 **Regola d'esame:** la risposta corretta è quasi sempre quella che rende l'errore **strutturato, classificato e azionabile**, con un **passo successivo** chiaro.

---

2. Errori di protocollo vs errori del tool

Tipo	Esempio	Come si comunica
Errore di protocollo	Tool inesistente, argomenti non validi rispetto allo schema	Errore JSON-RPC del protocollo MCP

Tipo	Esempio	Come si comunica
Errore di esecuzione del tool	API esterna giù, record non trovato, permesso negato	Risultato del tool con `isError: true` e contenuto descrittivo

Gli errori di esecuzione vanno restituiti **dentro il risultato** (`isError: true`) perché così arrivano **al modello**, che può ragionarci. Un'eccezione non gestita, invece, interrompe il flusso senza dare all'agente nulla su cui lavorare.

```
{
  "isError": true,
  "content": [{ "type": "text", "text": "
{\"errorCategory\":\"not_found\",\"isRetryable\":false,\"message\":\"Nessun
account con questa email.\"}" }]
}
```

---

3. Le categorie di errore

Categoria	Esempi	Ritentabile?	Reazione corretta
Transitorio	Timeout, reset TCP, HTTP 5xx, cache non raggiungibile	✅ Sì	Retry con backoff (meglio dentro il tool)
Capacità / rate limit	HTTP 429, servizio sovraccarico	✅ Sì, rispettando `retryAfter`	Attendere l'intervallo indicato e riprovare
Validazione	Formato data errato, campo mancante, ID malformato	❌ No	Correggere i parametri e richiamare
Non trovato	Nessun cliente/contratto/dataset corrispondente	❌ No	Chiedere all'utente di verificare o un altro identificativo
Autenticazione / permessi	Token scaduto, accesso negato	❌ No (non con lo stesso input)	Segnalare, rinnovare credenziali o scalare
Policy / business	Limite di credito superato, fattura già chiusa, licenza esclusa	❌ No	Spiegare all'utente il motivo, proporre alternative o scalare

> 💡 La domanda chiave è sempre: **riprovare la stessa identica richiesta può dare un esito diverso?** Se no, il retry è uno spreco.

---

4. Anatomia di una buona risposta di errore

```
{
  "isError": true,
  "errorCategory": "policy",
  "code": "CREDIT_LIMIT_EXCEEDED",
  "isRetryable": false,
  "message": "Il cliente ha già ricevuto il massimo di crediti di cortesia per
quest'anno.",
  "user_message": "Non posso applicare un ulteriore credito, ma posso inoltrare la
richiesta a un collega.",
  "suggested_action": "escalate_to_human",
  "correlation_id": "req-8f21c"
}
```

Campo	A cosa serve
`isError`	Dice al modello che l'operazione **non** è riuscita
`errorCategory` / `code`	Permette di **distinguere** casi che altrimenti sembrano identici
`isRetryable` (+ `retryAfterMs`)	Dice se e quando riprovare
`message`	Spiegazione leggibile della causa
`user_message`	Testo sicuro da mostrare al cliente (niente dettagli interni)
`suggested_action`	Il passo successivo consigliato
`correlation_id`	Tracciabilità per gli ingegneri, senza esporre dettagli interni al modello

Cosa **non** fare

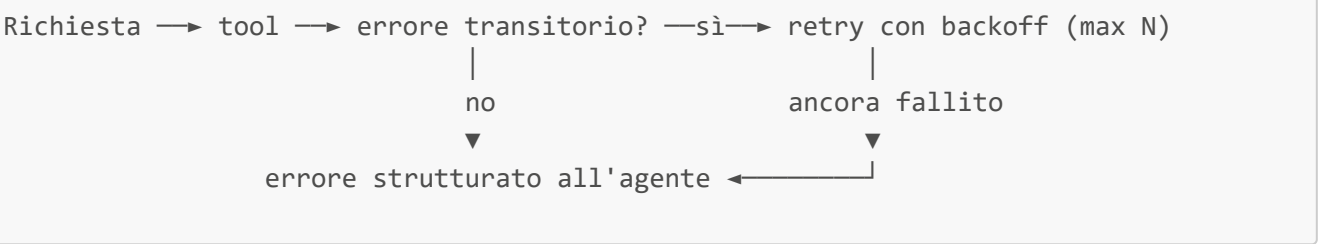
Risposta	Problema
`"error": "operazione fallita"`	Generica: l'agente non distingue un'interruzione da un rifiuto di policy
Solo classe dell'eccezione, URI interni, codici numerici della libreria	Diagnostica per ingegneri, inutile al modello
`isError: false` con risultato vuoto	**Nasconde** il fallimento: l'agente procede su dati mancanti
Un record segnaposto inventato (`"id": "temp-unknown"`)	L'agente lavora su dati falsi
Tutto convertito in "temporaneamente non disponibile"	Nasconde la causa reale e impedisce la correzione

Un esempio concreto: PDF illeggibile

✅ `isError: true`, causa probabile ("il documento sembra protetto da password"), file e fase coinvolti, suggerimento ("chiedi una copia sbloccata o il permesso di omettere la fonte").

5. Chi fa il retry: tool o agente?

Tipo di errore	Dove gestirlo	Perché
Transitorio (timeout, 5xx, 429)	**Dentro il tool**, con retry **limitati** e **backoff**	È un dettaglio tecnico: non serve sprecare turni e contesto dell'agente
Permanente (not found, validazione, auth, policy)	**Restituito all'agente** come errore strutturato	Serve ragionamento: correggere, chiedere, scalare



Regole:

- **Retry limitati**: mai all'infinito.
- **Backoff esponenziale**, rispettando `retryAfter` se presente.
- **Mai retry su errori permanenti**: un ID sbagliato resta sbagliato.
- Se dopo i retry il problema persiste, restituire un errore strutturato con `isRetryable: true`, così l'agente può decidere di riprovare più tardi o informare l'utente.

> 💡 Se l'errore restituito all'agente dice esplicitamente `isRetryable: true` con `retryAfterMs`, la reazione corretta è **attendere e riprovare**, non marcare subito il passo come fallito né saltarlo.

6. Il caso più insidioso: "nessun risultato"

Il servizio risponde **normalmente**, ma la query non trova nulla.

	Valutazione
Errore transitorio, da riprovare	❌ Riprovare la stessa email non cambierà nulla
Servizio sovraccarico, scusarsi per il disservizio	❌ Messaggio falso al cliente
Record segnaposto e prosecuzione	❌ Dati inventati
`not_found`, non ritentabile, chiedere all'utente di verificare o fornire un altro identificativo	✅

> ⚠️ Il caso opposto è altrettanto importante: se il servizio **non ha risposto** (es. 503), l'agente **non** deve dire "non esiste nessun ordine con questo numero". La ricerca non è stata eseguita: va detto, ritentato o segnalato.

7. Successo apparente: `isError: false` ma dati inaffidabili

A volte il tool dice di aver avuto successo, ma il contenuto è sospetto: ID di fonti inesistenti, etichette che contraddicono le note, campi nel formato sbagliato.

✅ L'agente che ha il contesto del compito tratta il risultato come **sospetto ma non fatale** e adatta la strategia: riesegue con input più mirati, chiede i passaggi di supporto, tiene solo i dati verificabili.

❌ Ripetere la stessa chiamata (darà lo stesso problema) · ❌ abortire l'intera esecuzione · ❌ riparare centralmente in un coordinatore che non ha il contesto.

> 💡 Se il controllo deve essere **garantito**, la soluzione diventa un hook `PostToolUse` di validazione (vedi gli appunti sugli hook).

---

8. Il ciclo lato API: `tool_result` e `is_error`

Con la Messages API, gli errori dei tool tornano al modello come blocco `tool_result`:

```
{
  "role": "user",
  "content": [{
    "type": "tool_result",
    "tool_use_id": "toolu_01A...",
    "is_error": true,
    "content": "not_found: nessun contratto C-90217. Chiedi al cliente di
verificare il numero o l'email dell'account."
  }]
}
```

Il ciclo dell'agente:

1. La risposta ha `stop_reason: "tool_use"` e contiene un blocco `tool_use`.
2. Il runtime esegue il tool con gli argomenti del blocco.
3. Il risultato (o l'errore, con `is_error: true`) viene aggiunto come `tool_result`.
4. Si richiama Claude, che decide il passo successivo.

> ⚠️ Nome e parametri del tool stanno nel blocco `tool_use`, **non** in `stop_reason`.

---

9. Trappole e domande da esame

Trappole

- ❌ Classificare "nessun risultato" come errore transitorio.
- ❌ Ritentare automaticamente errori di validazione, permessi o policy.
- ❌ Un unico messaggio di errore per casi diversi (limite superato, fattura chiusa, servizio giù).
- ❌ Restituire diagnostica interna invece di informazioni azionabili.
- ❌ Lasciare all'agente i retry di errori transitori che il tool può gestire da solo.